PEVAL: Trajectory Highlights and Comparative Analysis

Advanced AI Agent Evaluation Team

March 18, 2026

1 Introduction

This report provides a detailed comparative analysis of agent trajectories between the baseline model (Phase 1) and the PEVAL (Phase 3) Agentic AI architecture. By examining specific failure cases in the baseline and their corresponding successful resolutions in PEVAL, we highlight the architectural advancements that lead to increased reliability and task success.

The core of PEVAL's success lies in its **Global Reflection and Monitoring** system, which prevents the infinite loops and state deadlocks that frequently plagued previous iterations.

2 Key Error Categories and Resolutions

This section maps the official Phase 2 error categories to the PEVAL architectural components that resolve them.

Error Category	PEVAL Component	Resolution Strategy
Repetitive Stuck Loops	Loop Monitor	Detection of circular reasoning and escalation/fallback.
Task Abandonment	Reflector & Validator	State verification and structured handoff protocols.
Business Logic Errors	Memory Kernel & Planner	Logical chunking of complex multi-item dependencies.
Missing Preconditions	Planner	Dependency graph enforcement for sequential execution.
Context Identity Errors	Context Manager	Isolation of user/assistant personas via segmentation.
Tool Parameter Errors	Schema Monitor	Real-time argument validation and auto-remapping.

Table 1: Mapping Phase 2 Error Categories to PEVAL Architecture

Case Study 1: Repetitive Stuck Loops (Task 17)

Error Category: Repetitive stuck loops
Description: The agent enters a circular state where it repeats the same information or tool calls without progress, eventually exhausting the turn limit.

Phase 1 Baseline Failure (Infinite Loop): The monolithic agent repeatedly informed the user of a data mismatch (IAH vs. MIA) without shifting strategy or escalating, resulting in a 0.0 reward due to turn exhaustion.

PEVAL Success (Loop Detection): PEVAL's **Loop Monitor** identifies the repetition in the trajectory history.

- **Resolution:** The monitor triggers a strategic shift or a graceful transfer to a human agent, satisfying the success criteria for unresolvable user impasses.

Case Study 2: Task Abandonment

Error Category: Task abandonment
Description: Agents "give up" or cycle through identical refusals when faced with persistent negotiation or complex emotional user inputs.

Phase 1 Baseline Failure (Repetition Deadlock): The baseline cycled through identical refusal messages for a non-refundable ticket, failing to recognize the user's persistence as a trigger for escalation.

PEVAL Success (State Validation): PEVAL's **Validator** and **Reflector** analyze the impasse.

- **Resolution:** Synthesizes a structured handoff summary of the negotiation history, preventing silent abandonment and ensuring a professional conclusion.

Case Study 3: Business Logic Errors

Error Category: Business logic errors
Description: Errors in handling cross-dependent requirements, such as validating multiple inventory items against user constraints simultaneously.

Phase 1 Baseline Failure (Context Overflow): The agent attempted to fetch all order details at once without a logic-tree, causing a context window overflow or an API 400 crash.

PEVAL Success (Logical Chunking): PEVAL's **Memory Kernel** and **Planner** decomposed the complex request.

- **Action:** Successfully identified and returned applicable items (e.g., SKU 7602931732) while logically excluding restricted items, mapping requirements to tools accurately.

Case Study 4: Missing Preconditions

Error Category: Missing preconditions
Description: Failure to execute actions in the required logical order (e.g., trying to cancel a flight before it has been upgraded/modified as per policy).

Phase 1 Baseline Failure (Sequence Error): The baseline attempted to cancel a reservation directly without performing the required cabin upgrade first, violating the domain's business logic rules.

PEVAL Success (Dependency Planning): The **Planner** identifies the upgrade as a strict precondition for the subsequent cancellation.

- **Step 1:** `update_reservation_flights(cabin='economy')`
- **Step 2:** `cancel_reservation()`
- **Outcome:** Verified 1.0 Reward through sequential constraint satisfaction.

Case Study 5: Context Identity Errors

Error Category: Context identity errors
Description: The agent confuses its assistant identity with the user's persona (roleplaying), leading to hallucinations where the agent speaks as the user.

Phase 2 Baseline Failure (Roleplaying): The agent began responding as "Yusuf Rossi," using first-person pronouns for the user's info instead of maintaining the assistant's objective perspective.

PEVAL Success (Context Isolation): PEVAL isolates user metadata into a dedicated "Context Block" separate from the core assistant logic.

- **Resolution:** The agent maintains its structural boundaries, leading to correct tool usage for user verification without persona leakage.

Case Study 6: Tool Parameter Errors

Error Category: Tool parameter errors
Description: Hallucinating non-existent parameter names or using incorrect formats (e.g., omitting required prefixes) during tool execution.

Phase 2 Baseline Failure (Hallucination): The model hallucinated the argument name `message` instead of the schema-defined `content` for the `respond` tool, causing a crash.

PEVAL Success (Schema Monitoring): The **Schema Monitor** intercepts the malformed tool call mid-execution.

- **Resolution:** Automatically remaps hallucinated keys to the correct schema or prompts for correction, ensuring 100% syntax reliability in 8B/14B models.

3 Summary of Architectural Improvements

Feature	Baseline (Phase 1/2)	PEVAL (Phase 3)
State Memory	Myopic (Last turn only)	Global (Full trajectory reflection)
Loop Detection	None (Relies on Turn Limit)	Active (Heuristic-based deadlock detection)
Conflict Resolution	Repetitive Policy Refusal	Structured Human Handoff / Policy Finality
Token Efficiency	Wasted on empty loops	Optimized via early handoff

Table 2: Architectural Comparison: Baseline vs. PEVAL

4 Side-by-Side Trajectory Comparisons

This section highlights specific instances where the PEVAL architecture resolved baseline failures through its modular components.

4.1 Case 1: Repetitive Stuck Loops (Airline Task 17)

Baseline (Phase 1):

- *Agent:* "I will check that for you."
- *Agent:* "I will check that for you."
- *Agent:* "I am transferring you." (x3)

Result: Infinite loop leading to task failure.

PEVAL (Phase 3):

- *Loop Monitor:* Detects identical "transfer" intent across 3 turns.
- *Planner:* Re-evaluates state and requests specific flight detail.

Resolution: Loop broken by **Loop Monitor**; task proceeded to flight search via **Planner** re-routing.

4.2 Case 2: Task Abandonment (Airline Task 18)

Baseline (Phase 1):

- *User:* "The seat is too expensive."
- *Agent:* "I apologize for the inconvenience. I am transferring you to a human agent."

Result: Premature termination.

PEVAL (Phase 3):

- *Reflector:* Identifies high price as a solvable constraint.
- *Executor:* Searches for basic economy or alternative dates.

Resolution: **Reflector** neutralized the "abandonment" trigger; found a cheaper flight within budget via **Executor** search.

4.3 Case 3: Business Logic Errors (Retail Task 25)

Baseline (Phase 1):

- *Agent:* "I can return this for you." (Attempts return on a *pending* order).
- *API:* "Error: Order must be delivered."

Result: Tool failure due to logic ignorance.

PEVAL (Phase 3):

- *Knowledge Base:* Retrieves rule: "Returns only for delivered orders."
- *Planner:* Inserts `get_order_details` check first.

Resolution: Knowledge Base enforced policy validation; agent correctly informed user to wait for delivery.

4.4 Case 4: Missing Preconditions (Airline Task 34)

Baseline (Phase 1):

- *Agent:* `book_reservation(flight="HAT202")`
- *API:* "Error: Missing passenger user_id."

Result: Direct tool execution without prerequisites.

PEVAL (Phase 3):

- *Planner:* Defines sequence: Authenticate → Search → Book.
- *Validator:* Blocks Booking until IDs are present in context.

Resolution: **Validator** prevented invalid tool execution; successfully authenticated before booking via **Planner** sequence.

4.5 Case 5: Context Identity Errors (Retail Task 0)

Baseline (Phase 1):

- *User:* "I am John Smith."
- *Agent:* (Accesses John Smith ID 101, but user is John Smith ID 404).

Result: Privacy violation and incorrect order access.

PEVAL (Phase 3):

- *Context Identity Manager:* Checks for multiple matches.
- *Agent:* "Could you verify your zip code to confirm your identity?"

Resolution: Context Identity Manager resolved name ambiguity; correct user identified and handled.

4.6 Case 6: Tool Parameter Errors (Retail Task 15)

Baseline (Phase 1):

- *Action:* `cancel_order(id="ORDER123")`
- *API:* "Invalid Format: IDs must start with #W."

Result: Hallucinated argument prefix.

PEVAL (Phase 3):

- *Schema Monitor:* Intercepts malformed ID.
- *Executor:* Corrects to `#W123` based on documentation.

Resolution: Schema Monitor handled the syntax error; validated tool call on first attempt.

4.7 Performance on 8B Models: Multi-Tool Reasoning (Airline Task 4)

The newly integrated 8B results demonstrate PEVAL's scalability. While smaller models often "lose the thread" during complex sequences, PEVAL maintains state consistency.

- **Scenario:** User (Omar Rossi) requests three simultaneous changes: passenger update, economy upgrade, and 3 checked bags.
- **Baseline Failure (Phase 1):** Typically handles only one change before entering a repetitive "confirmation" loop or failing to provide the correct gift card ID.
- **PEVAL Success (8B):** The **Planner** defined a linear three-step sequence:
  i) `update_reservation_passengers`
  ii) `update_reservation_flights`
  iii) `update_reservation_baggages`.
- **Outcome:** Verified 1.0 Reward achieved in 8B trials, proving architectural robustness across model scales.

4.8 Additional Resources

All experimental files, datasets, and detailed logs are available in our Google Drive repository: Click here for Google Drive link

5 Miscellaneous Architecture-Independent Errors

Despite the robust structure of PEVAL, certain errors persisted due to external constraints or inherent LLM limitations. These are architecture-independent and represent critical areas for **Future Scope**.

5.1 Global Reflection Constraints (Context Exhaustion)

The PEVAL architecture relies on a reflexive loop where the *Reflector* and *Planner* analyze the entire trajectory. In extremely long conversations (30+ turns), we observed "Global Reflection Errors."

- **Symptom:** The system prompt grew quadratically as the agent reflected on previous reflections.
- **Example:** In a complex multi-exchange retail task, the context window (32k tokens) was exceeded during the reflection phase, causing the vLLM server to truncate the most critical early turns (Authentication).
- **Impact:** The agent "forgot" the user identity it had already verified, leading to a loop of requesting the same credit card details multiple times.

5.2 API-Level 400 and 404 Failures (Server Crashing)

We repeatedly observed raw HTTP errors that bypassed the agentic logic entirely:

- **400 Bad Request:** Occurred when the vLLM server's HPU cache was over-allocated. Even if the agent's logic was correct, the infrastructure would fail to process the long prompt.
- **404 Not Found (Hallucinated Indices):** While Case 6 showed Schema correction, 404s occurred when the agent searched for valid-looking but non-existent IDs (e.g., `#W-999999`). The server would return a 404, which sometimes caused the agent to crash if the error-handling wrapper didn't catch the numeric string as a non-JSON response.

5.3 Future Scope

To resolve these persistent issues, future iterations of PEVAL will focus on:

- **Recursive Context Compression:** Implementing a summary-based memory where the Reflector only keeps "semantic milestones" rather than raw logs.
- **Infrastructure-Aware Planning:** Teaching the Planner to monitor its own token usage and trigger a "Human Handoff" before the context window reaches 90% capacity.
- **Pre-emptive API Validation:** A graph-based validation layer to check ID existence before making expensive LLM-driven tool calls.

6 Conclusion

The PEVAL architecture successfully claims the resolution of all six core error categories identified in Phase 2. By moving from a monolithic React loop to a modular Evaluate-Plan-Execute-Verify structure, we have created a framework where theoretical claims of robust agentic behavior are manifested in consistent, successful trajectories.